

CITS5508 Practice Exam — Chapter 7 — ANSWER KEY

Question 1.

(a) (4 marks) By the **law of large numbers**, aggregating many predictors that are each **slightly better than random** drives the majority vote toward the correct answer — like a biased coin (51% heads) that, over many tosses, gives a heads majority with high probability (~75% over 1,000 tosses). So with enough sufficiently good, diverse weak learners, the **ensemble** can achieve high accuracy (a **strong learner**) even though each member is weak.

(b) (4 marks) Critical assumption: the predictors make **independent / uncorrelated errors**. (If they all make the same mistakes, majority voting just repeats them.) In practice they're trained on the same data so errors are correlated; to improve independence, **use very different training algorithms** (or train on different random subsets of data/features) so they make different kinds of errors.

(c) (4 marks) **Hard voting** predicts the **majority class** among the predictors. **Soft voting** averages the predicted **class probabilities** and predicts the argmax. **Soft voting usually performs better** because it gives **more weight to confident votes** (a predictor very sure of a class counts more than one that's barely above 50%). (Requires all predictors to expose `predict_proba`.)

Question 2.

(a) (3 marks) Both train each predictor on a **random subset** of the training set. **Bagging** samples **with replacement** (an instance can be drawn multiple times for the same predictor); **pasting** samples **without replacement**.

(b) (3 marks) Each predictor is trained on a different random subset, so they make **different errors**. Averaging (or majority voting) over many such predictors **cancels out** much of the individual variability/noise → the ensemble's predictions vary far less than a single predictor's → **lower variance** (bias stays similar).

(c) (4 marks) **OOB** instances are the training instances **not sampled** for a given predictor. Under bagging (sampling m with replacement), each predictor sees on average 63% of the instances, so **37%** are OOB for it. Because each instance is OOB for many predictors, those predictors can evaluate it → the ensemble gets a **free validation estimate with no separate holdout set** (`oob_score_`).

(d) (2 marks) In bagging/pasting the predictors are **independent**, so they can be trained (and queried) **in parallel** across cores/servers. **Boosting is sequential** — each predictor depends on the previous one — so it **cannot be parallelised** the same way.

Question 3.

(a) (4 marks) A **random forest** is an ensemble of **decision trees** trained via **bagging** (each on a bootstrap sample). The extra randomness: at each node, instead of searching for the best split among **all** features, it searches only within a **random subset of features** (default $\sqrt{n}$). This makes the trees more diverse (more bias, less variance) → usually a better overall model.

(b) (4 marks) **Extra-trees** add randomness by also using a **random threshold** for each candidate feature, instead of searching for the best threshold. Consequence: **more**

bias, lower variance, and much faster training (finding the optimal threshold per feature is the costliest step of growing a tree). Whether extra-trees beat a random forest is unknown a priori → cross-validate both.

(c) (4 marks) A random forest measures a feature's importance by **how much the nodes using that feature reduce impurity on average**, across all trees (weighted by the number of samples reaching each node), normalised to sum to 1 (feature_importances_). Use: quick **feature selection** / understanding which features matter.

Question 4.

(a) (3 marks) **Boosting** trains predictors **sequentially**, each one trying to **correct the errors** of its predecessor, then combines them. Fundamentally different from bagging: boosting is **sequential and dependent** (cannot parallelise) and focuses on hard cases, whereas bagging trains **independent** predictors in parallel on random subsets.

(b) (4 marks) **AdaBoost**: train a base predictor; **increase the weights of the instances it misclassified**; train the next predictor on the reweighted data; repeat. New predictors focus increasingly on the **hard cases**; the final prediction is a **weighted vote** (by each predictor's accuracy). If **overfitting**: **reduce the number of estimators** and/or **regularise the base estimator** more (also reduce the learning rate).

(c) (5 marks) **Gradient boosting** (e.g. GBRT): each new predictor is fit to the **residual errors** of the current ensemble, and predictions are the **sum** of all predictors. **learning_rate** scales each tree's contribution (**shrinkage**). A **low** learning rate makes each tree contribute less, so you need **more trees** to fit the training set, but the result usually **generalises better**. Too few trees → underfit; too many → overfit (tune via early stopping).

Question 5.

(a) (4 marks) A voting classifier uses a **fixed** aggregation rule (hard/soft voting). **Stacking** instead **trains a model to do the aggregation**: the base predictors' outputs become input features to a final model called the **blender** (or meta-learner), which learns how best to combine them into the final prediction.

(b) (4 marks) If the blender were trained on the base predictors' predictions for the **same data they were trained on**, those predictions would be **over-optimistic / over-fit** (the base models have effectively memorised those instances), so the blender would learn from **unrealistically good** inputs and generalise poorly. Using **cross-validated (out-of-sample)** predictions gives the blender **clean, realistic** inputs that reflect true generalisation.

(c) (4 marks)

```
from sklearn.ensemble import VotingClassifier, RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC

voting_clf = VotingClassifier(
    estimators=[
        ('lr', LogisticRegression(random_state=42)),
        ('rf', RandomForestClassifier(random_state=42)),
        ('svc', SVC(probability=True, random_state=42)),    # needed for soft voting
    ],
    voting='soft')
```

```
voting_clf.fit(X_train, y_train)
```

Note: SVC must be created with **probability=True** so it can produce the class probabilities that soft voting averages.